Using the ESP32 I2C Sniffer Sketches

A step-by-step guide to i2c_sniffer.ino, i2c_sniffer_temp_decode.ino, and the WiFi-controlled version

Overview

This project produced three ESP32 sketches, each building on the last. This guide explains what each one does, how to wire it, how to use it, and what output to expect - in the order they were built:

- i2c_sniffer.ino - the general-purpose passive I2C bus sniffer. Prints every raw transaction on the bus.
- i2c_sniffer_temp_decode.ino (early version) - the same sniffer, but automatically detects the temperature-carrying frame and prints a computed reading.
- i2c_sniffer_temp_decode.ino (final version) - all of the above, plus an electronic button-press trigger and a WiFi-based control page.

All three share the same underlying sniffing technique, so the wiring is identical across all of them. Only the software behavior differs.

Common Wiring (applies to all three sketches)

1. Locate the I2C bus you want to observe - either the EEPROM's SDA/SCL lines, or the sensor chip's SDA/SCL lines, depending on which layer you're investigating.
2. Tap SDA and SCL in parallel with jumper wires. Do not cut the existing trace - the sniffer only listens, it never drives the bus.
3. Check the bus voltage with a multimeter first. If it runs at 3.3V, connect directly to the ESP32. If it runs at 5V, use a logic-level shifter or resistor divider on both lines to protect the ESP32's GPIOs.
4. Connect ESP32 GND to the thermometer's GND. This common ground is required for the sniffer to read the signals correctly.
5. By default the sketches use GPIO21 for SDA and GPIO22 for SCL (PIN_SDA / PIN_SCL near the top of each file) - change these if you wire to different pins.

1. i2c_sniffer.ino - the raw bus sniffer

What it does

This is the foundational sketch. It configures two GPIOs as plain inputs and uses interrupts to reconstruct every I2C transaction happening on the bus, printing each one in full: the address being talked to, whether it's a read or a write, and every data byte along with its ACK/NACK bit.

How it works internally

- An interrupt on the SCL pin's rising edge samples the SDA line at that exact instant - this is when a data bit is guaranteed to be valid on a real I2C bus.
- An interrupt on the SDA pin's change only acts when SCL is currently HIGH: SDA falling while SCL is high means a START condition; SDA rising while SCL is high means a STOP condition.
- Every 9 sampled bits (8 data bits plus 1 ACK/NACK bit) are assembled into a byte and pushed into a small ring buffer.
- The main loop() continuously drains that ring buffer and prints readable FRAME blocks.

Step-by-step usage

6. Open i2c_sniffer.ino in the Arduino IDE.
7. Set PIN_SDA and PIN_SCL at the top of the file to match your wiring (default: 21 and 22).
8. Optionally set FILTER_ADDR to a specific 7-bit address if you only want to see traffic for one device; leave it as -1 to log everything on the bus.
9. Select your ESP32 board and COM port in the Arduino IDE, then upload.
10. Open Serial Monitor and set the baud rate to 921600 (this sketch prints a lot of data, so a fast baud rate avoids dropped bytes).
11. Trigger a reading on the thermometer (press its button, or place it near a warm object) and watch the transactions appear.

Example output

```
=== FRAME WRITE addr=0x10 ===
  byte[0] = 0x26  (ACK)
  byte[1] = 0x69  (ACK)
  byte[2] = 0x0F  (ACK)
--- STOP ---
```

Use this sketch whenever you need to explore a bus you don't understand yet - it is the tool for discovery, showing everything so you can identify which addresses and byte patterns matter.

2. i2c_sniffer_temp_decode.ino - automatic temperature decoding

What it does

Once the meaningful frame pattern was identified (a write to register 0x26 carrying a 2-byte little-endian raw value), this version adds logic on top of the same sniffer to recognize that pattern automatically, convert it to a temperature, and print just the result - instead of requiring you to manually read raw bytes and do the math yourself.

How it works internally

- The sniffing core (interrupts, ring buffer, frame reconstruction) is identical to i2c_sniffer.ino.
- Each completed frame's bytes are captured into a small buffer (frameBytes[]).
- When a frame's STOP condition arrives, printTempIfApplicable() checks whether byte[0] was 0x26 (the register address that carries the raw temperature).
- If the raw 16-bit value falls in the 0x7Fxx range, it is recognized as the sensor's error/out-of-range sentinel and printed as an error instead of a bogus temperature - this is what corresponds to the thermometer showing “HI”.
- Otherwise, the raw value is converted using degrees_C = raw / 100 and degrees_F = degrees_C * 9 / 5 + 32, and only a single clean line is printed.

Step-by-step usage

12. Open i2c_sniffer_temp_decode.ino in the Arduino IDE.
13. Confirm PIN_SDA and PIN_SCL match your wiring, same as before.
14. Upload to the ESP32, then open Serial Monitor at 921600 baud.
15. Trigger a reading on the thermometer.
16. Read the single output line for that reading.

Example output

```
TEMPERATURE : 102.3 F
```

or, if the sensor returned its error sentinel:

```
TEMPERATURE : HI (error)
```

Important limitation to remember

The raw/100 formula was confirmed accurate for normal body-temperature readings (roughly 96-103 F), but it underestimated a 109.2 F reading by about 2.3 F. This means the real device applies extra compensation at higher temperatures that this simple formula does not yet reproduce. Treat any computed value above about 103 F as a likely underestimate until that compensation curve is captured.

3. WiFi-Controlled Version - remote triggering and browser display

This is the final evolution of i2c_sniffer_temp_decode.ino. It keeps all of the sniffing and temperature-decoding behavior above, and adds two new capabilities: triggering a reading electronically (without pressing the physical button), and controlling everything from a web browser over WiFi instead of a wired serial connection.

Part A - Electronic button trigger

Before WiFi was added, the sketch first gained the ability to simulate a physical button press using a single GPIO pin.

How it works

- A momentary push button has two legs: one wired to GND, and one wired to a GPIO on the thermometer's own MCU that is normally held HIGH by a pull-up resistor. Pressing the button briefly connects that GPIO to GND.
- An ESP32 GPIO (PIN_BUTTON, default GPIO4) is wired to that same MCU-side pad. It sits in high-impedance INPUT mode by default, so it has no effect on the button circuit while idle - the physical button keeps working normally.
- When triggered, the firmware briefly switches that pin to OUTPUT, drives it LOW for about 150 milliseconds (PRESS_MS), then switches it back to INPUT - electrically identical to a human pressing and releasing the button.

Wiring the button trigger

17. Find the thermometer's push button on the PCB and identify its two solder pads.
18. Use a multimeter in continuity mode: touch one probe to a known ground point, then touch the other probe to each button leg. The leg that shows continuity to ground is the ground pad; the other is the MCU-side pad.
19. Wire the ground pad to ESP32 GND (already shared from the I2C wiring).
20. Wire the MCU-side pad to the ESP32 pin defined by PIN_BUTTON (GPIO4 by default).

Triggering a press manually (serial)

21. Open Serial Monitor, ensure it can send text (not just receive).
22. Type the letter S and press Enter.
23. You should see [button] simulated press in the log, followed by the usual sniffed frames and a TEMPERATURE line, exactly as if the physical button had been pressed.

Part B - WiFi control page

The final addition lets you trigger readings and see results from any phone or computer on the same WiFi network, with no USB cable required.

How it works

- On startup, the ESP32 connects to the WiFi network using the credentials defined near the top of the file (WIFI_SSID / WIFI_PASSWORD) and prints the IP address it was assigned to Serial Monitor.
- A lightweight web server (the WebServer library) is started with three endpoints: the root page (/), a /trigger endpoint that fires the same simulated button press described above, and a /status endpoint that returns the most recently computed temperature as plain text.
- The root page is a small HTML/JavaScript page: it polls /status every two seconds to keep the displayed temperature current, and has a “Take Reading” button that calls /trigger.

Step-by-step usage

24. Open the sketch and set WIFI_SSID and WIFI_PASSWORD to your network's credentials.
25. Wire the button-trigger pin as described in Part A (the I2C sniffing wiring stays the same as before).
26. Upload to the ESP32 and open Serial Monitor at 921600 baud.
27. Wait for the log to show Connecting to WiFi..., followed by Connected! Open this in your browser: http://<ip address>.
28. On a phone or computer connected to the same WiFi network, open that address in a browser.
29. Click the “Take Reading” button on the page. This triggers the reading electronically, and the displayed temperature updates automatically within a couple of seconds.

Typing S into Serial Monitor still works at the same time - both control paths (serial and WiFi) are active simultaneously, so you can use whichever is more convenient.

Security note

The WiFi password is stored in plain text near the top of this sketch for convenience, since it only controls a piece of hardware on a private home network. Avoid sharing this specific file publicly (forums, GitHub, etc.) without removing or replacing the credentials first.

Quick Reference

File	Purpose	Key output
i2c_sniffer.ino	Raw bus explorer - see every transaction	Full FRAME / byte / ACK log
i2c_sniffer_temp_decode.ino (early)	Auto-detect and convert the temperature frame	"TEMPERATURE : xx.x F" line
i2c_sniffer_temp_decode.ino (final, with WiFi)	Same decoding, plus remote trigger via serial or browser	Same temperature line, plus a live web page

Together, these three sketches represent the natural progression of this project: first understand the bus, then automate reading the result, then automate triggering and controlling the whole device remotely.